# Lecture 9. Pipes

Alternatives to the pipe, when not to use it, other magrittr tools

## Contents


**9.1 Why pipes** **2**

**9.2 When not to use the pipe** **3**

**9.3 Other tools from magrittr** **3**

## 9.1 Why pipes

**Motivation.** This lecture starts the programming part of the course. Code is a means of communication, with the computer and with the people (including your future self) who will read it. The pipe is the first tool for writing code that reads in the same order as the analysis it performs.

The pipe `%>%` comes from the `magrittr` package; tidyverse packages load it automatically. To see why it helps, write the same sequence of actions four ways, using the nursery rhyme "Little bunny Foo Foo went hopping through the forest, scooping up the field mice and bopping them on the head".

**Save each intermediate step.**

```
foo_foo <- little_bunny()
foo_foo_1 <- hop(foo_foo, through = forest)
foo_foo_2 <- scoop(foo_foo_1, up = field_mice)
foo_foo_3 <- bop(foo_foo_2, on = head)
```

The code is cluttered with unimportant names, and the suffixes must be incremented carefully (it is easy to use `foo_foo_1` where `foo_foo_2` was meant). Worry about memory is usually misplaced: R shares columns between modified copies of a data frame, so copies are cheap until a column changes.

**Overwrite the original.**

```
foo_foo <- hop(foo_foo, through = forest)
foo_foo <- scoop(foo_foo, up = field_mice)
foo_foo <- bop(foo_foo, on = head)
```

Debugging is painful (a mistake means rerunning from the start), and the repeated name hides what changes on each line.

**Function composition.**

```
bop(scoop(hop(foo_foo, through = forest), up = field_mice), on = head)
```

The code must be read inside out and right to left, and each function's arguments end up far from its name (the "Dagwood sandwich" problem).

**The pipe.**

```
foo_foo %>%
  hop(through = forest) %>%
  scoop(up = field_mice) %>%
  bop(on = head)
```

The code reads as a sequence of verbs, "Foo Foo hops, then scoops, then bops". Behind the scenes magrittr rewrites it as a function with a placeholder `.`, roughly

```
my_pipe <- function(.) {
  . <- hop(., through = forest)
  . <- scoop(., up = field_mice)
  bop(., on = head)
}
```

Consequently the pipe does not work with functions that use the current environment (`assign()`, `get()`, `load()`) or rely on lazy evaluation in a special way (`tryCatch()`, `try()`, `suppressMessages()`).

**The native pipe [Supplement]**

Since R 4.1 base R has its own pipe `|>`, which works like `%>%` in simple cases: `x |> f(y)` is `f(x, y)`. It is slightly faster and needs no package, but it has no `.` placeholder in the same form (R 4.2 added `_` for named arguments) and none of the special pipes below. These notes

use `%>%`, as the course does.

## 9.2 When not to use the pipe

Pipes are for a linear sequence of operations on one object. Use something else when

- the pipe is longer than about ten steps: create intermediate objects with meaningful names, which also makes debugging easier;
- there are several inputs or outputs, i.e. no single primary object is being transformed;
- the steps form a directed graph with complex dependencies rather than a chain.

## 9.3 Other tools from magrittr

**The tee pipe** `%T>%` returns its left-hand side instead of the right-hand side, so a step with a side effect (plotting, printing, saving) does not end the pipeline. Without it, `plot()` returns `NULL` and the next step receives nothing:

```
rnorm(100) %>%
  matrix(ncol = 2) %>%
  plot() %>%
  str()
```

```
##  NULL
```

```
rnorm(100) %>%
  matrix(ncol = 2) %T>%
  plot() %>%
  str()
```

```
##  num [1:50, 1:2] -0.387 -0.785 -1.057 -0.796 -1.756 ...
```

**The exposition pipe** `%$%` "explodes" the variables of a data frame so they can be referred to by name in functions without a `data` argument:

```
mtcars %$% cor(disp, mpg)
```

```
## [1] -0.8475514
```

**The assignment pipe** `%<>%` replaces `mtcars <- mtcars %>% transform(cyl = cyl * 2)` by `mtcars %<>% transform(cyl = cyl * 2)`. Many prefer to avoid it: assignment is a special operation that should be clearly visible at the start of a line.